

ENGLISH FOR WORK / SEPTEMBER 2026

Software Development English



Conversation lab

8 complete workplace dialogues, followed by eight additional role-play cases. Study the exchanges, then make the conversation your own.

PRACTICE THAT TRANSFERS TO WORK

Read a realistic case. Study complete conversations. Find the right language, speak, get feedback, and try again.

Eight lessons | Intermediate to advanced | Independent or classroom study

For Software developers, application engineers, code reviewers, and maintainers.

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

All scenarios, figures, and model responses are fictional teaching material. Use invented details in practice.

[Open this course on English Ladder](#)

Inside this conversation lab

Original fictional training conversations in professional English. These are realistic scripts, not transcripts of actual people. The professional roles are the speaker labels; all figures, incidents, and organizations are invented.

- 01 Pagination without a promise about order
- 02 Reducing the example without losing the failure
- 03 A broader cleanup hides the behavior change
- 04 The small upgrade with a changed default
- 05 A regression assertion tied to the wrong detail
- 06 The cache proposal needs a freshness contract
- 07 An all-or-nothing claim needs a stated boundary
- 08 A migration handover with an untested older reader

Then try eight independent role-play cases, followed by feedback criteria and references.

Read it. Hear it. Make it yours.

1. Read the setting. Identify the roles, the immediate problem, and what each speaker needs.
2. Read the whole conversation aloud with a partner. In a group, give a third person the observer role. For three-speaker scripts, assign each professional a separate voice.
3. Notice how the speakers ask a specific question, qualify a claim, disagree, explain a technical point, or agree on a next step. Underline language you would actually use.
4. Cover the script. Recreate the exchange using the situation and professional roles. Keep the meaning, but change the wording.
5. Switch roles and introduce a complication: a missing record, a different priority, an unavailable colleague, or a changed assumption. End with a clear outcome or unresolved question.

Register and terminology

These colleagues use concise professional language. In an internal technical meeting, familiar abbreviations can be natural. With a new colleague or a client, expand unfamiliar terms and check understanding. Keep disagreement directed at the evidence or proposal, and match the degree of certainty to what is known.

Independent study

Speak each role aloud. Pause before the reply and supply your own response before revealing the next line. Record a second version using invented details, then compare its clarity and tone with the script.

Professional context

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

FULL DIALOGUE 01 / 12 SPEAKING TURNS

Pagination without a promise about order

Fictional developers discover an assumption in a consumer of a paginated listing.

Roles: Consumer developer / Service developer

Consumer developer: Our client uses each page's last item to predict the next page's first.

Service developer: Which ordering guarantee are you relying on? The current contract provides a continuation token, not alphabetical order.

Consumer developer: The sample response happened to be alphabetical, so I assumed that ordering was part of the interface.

Service developer: That example shows possible data, not a guarantee. The continuation token is the stated way to request more results.

Consumer developer: Then sorting each page locally would not establish a consistent order across the entire collection, would it?

Service developer: Not by itself. We'd need an agreed global ordering contract before promising that behavior to the caller.

Consumer developer: I'll remove the prediction based on the last label and preserve the supplied token for continuation.

Service developer: Please also distinguish reaching the end from a failed request. The contract gives them separate result shapes.

Consumer developer: I see that now: an absent next token ends this listing; an error result reports a request failure.

Service developer: Yes. Neither state should be inferred solely from whether the visible page looks short.

Consumer developer: We've identified two assumptions to remove: alphabetical order and page length as proof of completion.

Service developer: That narrows the adapter change and leaves any new ordering guarantee for an explicit interface discussion.

Notice the professional language

'Relying on' asks which guarantee a design depends on.

'Not by itself' rejects an insufficient inference without claiming the desired behavior is impossible.

Rehearse this dialogue with an AI partner

FULL DIALOGUE 02 / 12 SPEAKING TURNS

Reducing the example without losing the failure

Fictional developers decide which details belong in a useful reproduction.

Roles: Developer / Maintainer

Developer: I reduced the eighty-row report, with its unrelated fields, to two rows.

Maintainer: Does the reduced file still fail in the same observable way, or merely produce some other error?

Developer: It still duplicates the second label. The failure is incorrect displayed content, not an exception.

Maintainer: Then the expected and actual labels matter more than an unrelated stack trace. Which fields did you keep?

Developer: I kept identifier, label, and order. Removing the repeated identifier makes the duplication disappear in this example.

Maintainer: That supports investigating repeated identifiers, but don't assume the application prohibits them without checking the contract.

Developer: The contract allows the same identifier in different groups. I removed group while reducing the file, though.

Maintainer: Then that removal may have changed the meaning. Can you restore group and see whether the example remains representative?

Developer: With two different groups restored, it still duplicates the label. Both groups should retain their own label.

Maintainer: Now the example preserves the allowed input condition as well as the failure, which makes it more useful.

Developer: I'll attach those two rows, their group values, and the expected separate labels to the review.

Maintainer: That gives us a small, valid example instead of a smaller file that accidentally tests a different contract.

Notice the professional language

stack trace - A record of function calls associated with a point in execution, often captured when an error occurs.

Rehearse this dialogue with an AI partner

FULL DIALOGUE 03 / 12 SPEAKING TURNS

A broader cleanup hides the behavior change

Fictional developers agree how to make a reviewable revision without losing completed work.

Roles: Reviewer / Author

Reviewer: This pull request changes retry behavior and reformats three large files. I'm struggling to isolate the behavioral difference.

Author: The formatting tool touched those files while I was editing. The actual retry change is only twelve lines.

Reviewer: Could you separate the formatting change so the retry review has a clear before-and-after comparison for those lines?

Author: I can preserve the formatting work in a separate proposal. Which retry question should this review focus on?

Reviewer: The new branch retries on every error result, whereas the contract identifies only one result as eligible.

Author: Then the issue isn't merely diff size. The condition is broader than the specified behavior as well.

Reviewer: Exactly. Please narrow the condition to the stated result and show how the other error results are handled.

Author: I'll keep them unchanged and include examples distinguishing the eligible result from the ordinary validation failure.

Reviewer: That makes the behavior legible without hundreds of unrelated whitespace changes.

Author: I've separated formatting and limited this revision's description to the retry condition and its evidence.

Reviewer: Thanks. I can now review that decision on its own, and the formatting work hasn't been discarded.

Author: Once the focused change is reviewed, we'll consider the separate cleanup without conflating their approval status.

Notice the professional language

pull request - A proposal to incorporate a set of code changes, commonly accompanied by a diff and review discussion.

Rehearse this dialogue with an AI partner

FULL DIALOGUE 04 / 12 SPEAKING TURNS

The small upgrade with a changed default

Fictional maintainers examine an upgrade note before calling an update behavior-neutral.

Roles: Developer / Reviewer

Developer: The package update looks small, but its note changes the parser's default mode.

Reviewer: Which caller sets a mode explicitly, and which leaves that option unspecified?

Developer: Our main caller sets strict mode explicitly. A background import leaves the option out and uses the default.

Reviewer: Then the callers have different exposure to the changed default. Have we compared the actual malformed-input examples?

Developer: Not yet. The note names the new default, but it doesn't prove our import's outputs remain equivalent.

Reviewer: Please avoid a blanket no-impact statement. Could you isolate the import caller and its relied-on error behavior?

Developer: Yes. I'll show its input examples and compare expected results under the documented old and new modes.

Reviewer: Also distinguish this package's documented change from assumptions about what a small version increment normally implies.

Developer: Agreed. The relevant evidence is the stated default and our omitted option, not the size of the version label.

Reviewer: Would explicitly selecting a mode be a possible proposal, subject to checking that mode's contract in the new version?

Developer: Possibly. I'll present that option with evidence, not silently change the caller while claiming only an upgrade.

Reviewer: Then the review can consider the dependency and caller behavior together, with the affected import clearly identified.

Notice the professional language

'Different exposure' explains why one dependency change can affect callers differently.

'A possible proposal, subject to...' offers an option without claiming compatibility before checking it.

Rehearse this dialogue with an AI partner

FULL DIALOGUE 05 / 12 SPEAKING TURNS

A regression assertion tied to the wrong detail

Fictional developers review a test that fails after harmless internal restructuring.

Roles: Developer / Reviewer

Developer: The regression test fails after the helper rename, although the displayed error remains correct.

Reviewer: What does the assertion observe? Is it checking the user-visible result or the old helper's internal name?

Developer: It checks that the old helper is called once. I used that as a shortcut for checking the message.

Reviewer: Then the test may reject a harmless refactor while missing a wrong message produced through the same helper.

Developer: So the stable requirement is the rejection message, not which internal function happens to produce it.

Reviewer: Exactly. In this case, the contract names the outcome and doesn't require a particular internal call structure.

Developer: I'll assert the visible result for the rejected save and keep the successful save as a separate example.

Reviewer: Would there be any reason to check call count here, such as a specified external side effect?

Developer: Not for this formatter. Another focused check already covers the storage interaction.

Reviewer: Then removing that internal-name dependency should make the regression case better aligned with its actual purpose.

Developer: I've revised the assertion. It now fails if success is shown after rejection, regardless of the formatter's name.

Reviewer: That preserves the defect guard while allowing implementation details to change without creating a false alarm.

Notice the professional language

regression test - A test intended to detect unintended changes in previously working or corrected behavior.

side effect - An observable effect beyond a function's returned value, such as changing stored state or sending a notification.

Rehearse this dialogue with an AI partner

FULL DIALOGUE 06 / 12 SPEAKING TURNS

The cache proposal needs a freshness contract

Fictional developers compare a faster response with an unresolved data-freshness requirement.

Roles: Developer / Reviewer

Developer: The prototype cache lowers median response time in our test. Could we adopt it for the availability display?

Reviewer: What freshness is required? Returning older values quickly may change the feature's meaning.

Developer: The current contract doesn't state how old the displayed value may be. The prototype can reuse a previous result.

Reviewer: Then latency isn't the only criterion. We need agreement about acceptable age before calling the cache behavior equivalent.

Developer: The benchmark also uses repeated identical requests, which makes reuse more common than in some real workflows.

Reviewer: Please make that workload explicit. It supports a result for repeated requests, not every pattern of use.

Developer: Would you accept a comparison showing response time, memory use, and the age of reused values together?

Reviewer: Yes, provided the freshness requirement is identified as unresolved rather than silently selected by the implementation.

Developer: I'll ask the interface owner about that requirement and label the prototype's reuse behavior in the comparison.

Reviewer: Also distinguish this design discussion from a claim that any particular cache library guarantees the required behavior.

Developer: Understood. The prototype supports a design comparison, but doesn't settle our production configuration.

Reviewer: Then we can evaluate the speed benefit alongside what users would give up in freshness and resource use.

Notice the professional language

'Equivalent' concerns behavior, not merely similar output shape.

'Alongside what users would give up' makes a tradeoff explicit without dismissing the measured benefit.

Rehearse this dialogue with an AI partner

FULL DIALOGUE 07 / 12 SPEAKING TURNS

An all-or-nothing claim needs a stated boundary

Fictional developers examine what a proposed transaction includes and excludes.

Roles: Reviewer / Developer

Reviewer: The design says all-or-nothing. Which effects are included in that claim, and which are outside it?

Developer: The proposed transaction groups the two local state updates. Sending a confirmation message occurs afterward through another component.

Reviewer: Then the phrase could mislead readers into thinking the state updates and message delivery share the same guarantee.

Developer: You're right. The transaction boundary covers the local updates; the message's delivery outcome is a separate concern.

Reviewer: If local updates complete but the message reply is lost, what can the caller actually conclude?

Developer: Only that message acknowledgment is missing. The caller cannot infer that the local updates were undone.

Reviewer: Does retrying the entire operation necessarily preserve the intended invariant, or is that repetition contract still unspecified?

Developer: It's unspecified. We need to distinguish repeating message delivery from requesting the state change as a new action.

Reviewer: Please revise the design explanation to name both boundaries rather than extending the all-or-nothing label across components.

Developer: I'll describe the local transaction separately and present the uncertain message outcome as an open interaction question.

Reviewer: That helps reviewers evaluate the guarantees without assuming particular storage or messaging behavior.

Developer: And it avoids offering one technical word as proof that every partial-outcome case has already been handled.

Notice the professional language

'Which effects are included?' asks for the scope of a guarantee.

'The caller cannot infer...' separates a missing acknowledgment from an unobserved state change.

Rehearse this dialogue with an AI partner

FULL DIALOGUE 08 / 12 SPEAKING TURNS

A migration handover with an untested older reader

Fictional maintainers review an additive data change and an overconfident recovery claim.

Roles: Developer / Maintainer

Developer: The schema migration adds an optional label field. I called it additive and said rollback was safe.

Maintainer: Additive describes the shape change, but have we checked how the older application reads records containing the new field?

Developer: The new build reads them successfully; the older reader hasn't been tested against them.

Maintainer: Then the evidence establishes new-build reading only. Optional for one reader doesn't guarantee tolerance in every other reader.

Developer: That's a useful distinction. I'll remove the safe-rollback claim and identify older-reader compatibility as the open check.

Maintainer: Please also state whether any existing field values were transformed, because that would create another compatibility question.

Developer: Existing values are unchanged in this migration. The only specified change is the added optional field.

Maintainer: Good. That narrows the review, but it still doesn't replace evidence about the older reader's actual behavior.

Developer: I'll hand over the migration, example records, and exact old and new artifact references.

Maintainer: I accept that compatibility review. Has production been changed, or does the handover concern staging only?

Developer: Staging only. Production rollout remains undecided, and the release note will say that explicitly after this correction.

Maintainer: Then we have a bounded review: an additive staging change, unchanged existing values, and one unverified older-reader interaction.

Notice the professional language

schema migration - A controlled change to the structure or organization of stored data.

Rehearse this dialogue with an AI partner

Eight more situations to make your own

The following cases are open role plays. They give you facts, contrasting roles, useful expressions, and a short model response. Use what you learned from the complete dialogues to create a longer exchange.

Preparation: two minutes. Conversation: three to five minutes. Feedback: one minute. Switch roles and repeat with the second-round challenge.

ROLE-PLAY CASE 01

Clarifying interface contracts

SHARED CASE

A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

Role A | Responding professional

Resolve an interface ambiguity using concrete input and outcome pairs. Use only the facts given. Choose two relevant terms and be ready to explain one of them in ordinary words.

Role B | Listener

Client developer: you want one way to clear a nickname, and your current client can send either an empty string or null.

Useful expressions

For input ___, the contract specifies ___.

Does ___ mean ___, or should it be rejected?

That would change the behavior expected by ___.

Cover this until after your first attempt

The contract already distinguishes an omitted nickname from an empty string: one leaves the value unchanged and the other clears it. Could we agree what null means before the client starts sending it? At present, null is unspecified, so I cannot describe it as another supported way to clear the field.

Second round

A colleague calls omission and an empty string interchangeable. Restate their different outcomes in this fictional contract.

Debrief and extend with AI

Which phrase helped the listener? Which case fact needed clarification? Say one better follow-up question, then repeat the exchange.

ROLE-PLAY CASE 02

Debugging from reproducible evidence

SHARED CASE

A fictional import fails on a three-row file containing an empty date field. A two-row file without that row succeeds. The stack trace points to date parsing, but no controlled test has isolated the empty field as the cause.

Role A | Responding professional

Explain a debugging hypothesis and propose evidence that distinguishes it. Use only the facts given. Choose two relevant terms and be ready to explain one of them in ordinary words.

Role B | Listener

Developer receiving the bug: ask which inputs and conditions differ between the passing and failing examples.

Useful expressions

The failure occurs with ___, but ___ also differs.

The trace points to ___; that does not yet establish ___.

A comparison holding ___ constant would help distinguish ___.

Cover this until after your first attempt

The failing three-row file contains an empty date, and the stack trace reaches date parsing. Those observations make the empty field a useful hypothesis, not yet a demonstrated cause. Could we compare otherwise identical examples with and without that value? The smaller passing file also changed the row count, so it does not isolate the field.

Second round

The same failing file succeeds once after a retry. Explain why one success does not establish a fix.

Debrief and extend with AI

Which phrase helped the listener? Which case fact needed clarification? Say one better follow-up question, then repeat the exchange.

ROLE-PLAY CASE 03

Giving actionable code-review feedback

SHARED CASE

A fictional pull request is titled 'rename export helper.' Its diff also moves a notification call outside a success branch, so failed exports would now notify the user as if complete. The naming change is acceptable; the behavior change is unintended.

Role A | Responding professional

Give precise review feedback that separates an acceptable edit from an unintended behavior change. Use only the facts given. Choose two relevant terms and be ready to explain one of them in ordinary words.

Role B | Listener

Author: the rename is intentional, but moving the notification was accidental. Ask which path needs a check.

Useful expressions

The change moves ___, which means ___.

My concern is ___, not ___.

Could you preserve ___ and add evidence for ___?

Cover this until after your first attempt

The helper rename is clear. In the same diff, the notification call moves outside the success branch, which would also notify after a failed export. Could you keep that call conditional on success and add a check for the failure path? My concern is the changed side effect, not your choice of helper name.

Second round

A second reviewer adds an optional naming preference. Label it optional while keeping the failure-path correction clearly required.

Debrief and extend with AI

Which phrase helped the listener? Which case fact needed clarification? Say one better follow-up question, then repeat the exchange.

ROLE-PLAY CASE 04

Requesting dependency evidence

SHARED CASE

A fictional application directly uses package ParcelKit, which brings in TextCore indirectly. A proposed ParcelKit upgrade changes the resolved TextCore version. The change note mentions a deprecated parsing entry point; the team's usage and replacement compatibility are unknown.

Role A | Responding professional

Ask for evidence that makes a dependency upgrade reviewable. Use only the facts given. Choose two relevant terms and be ready to explain one of them in ordinary words.

Role B | Listener

Maintainer: you have the change note and resolved dependency list, but have not searched the application's call sites yet.

Useful expressions

Which dependency introduces ___?

Could you link the change note and identify our use of ___?

Does the resolved version change from ___ to ___?

Cover this until after your first attempt

Could you identify whether our application or ParcelKit calls the deprecated parsing entry point, and link the relevant upgrade note? The proposed change also alters the resolved TextCore version, so please show the dependency path and lockfile difference. We need that evidence before calling this a drop-in update with no caller changes.

Second round

The package name is unchanged. Explain why that alone does not establish identical behavior across versions.

Debrief and extend with AI

Which phrase helped the listener? Which case fact needed clarification? Say one better follow-up question, then repeat the exchange.

ROLE-PLAY CASE 05

Explaining what a test actually exercises

SHARED CASE

A fictional save function's unit test replaces storage with a test double that always reports success. It checks the success message but never exercises a storage rejection. A previous bug incorrectly showed success after rejection; the real storage adapter is not tested here.

Role A | Responding professional

Explain what a test verifies and what its substitutes leave outside scope. Use only the facts given. Choose two relevant terms and be ready to explain one of them in ordinary words.

Role B | Listener

Reviewer: ask which collaborator is substituted and what assertion would fail if the success message reappeared on rejection.

Useful expressions

This test exercises __ while replacing __.

It asserts __, but it does not cover __.

A separate test is needed to examine __.

Cover this until after your first attempt

This unit test checks the save function's success path using a controlled substitute for storage. Because that substitute always succeeds, it does not exercise the rejection that triggered the earlier bug. A regression case should check the visible result when rejection is simulated. An integration test of the real adapter answers a different question.

Second round

A teammate proposes checking only that no exception was thrown. Explain why the visible message needs its own assertion.

Debrief and extend with AI

Which phrase helped the listener? Which case fact needed clarification? Say one better follow-up question, then repeat the exchange.

ROLE-PLAY CASE 06

Comparing performance tradeoffs fairly

SHARED CASE

In a fictional controlled benchmark on the same input and machine, implementation A has median latency 40 milliseconds and memory use 80 megabytes; B has 25 milliseconds and 160 megabytes. The available memory budget is 120 megabytes. Tail latency and production workloads are unmeasured.

Role A | Responding professional

Describe a measured benefit and its cost without generalizing beyond the benchmark. Use only the facts given. Choose two relevant terms and be ready to explain one of them in ordinary words.

Role B | Listener

Reviewer: prioritize the fixed memory budget and ask whether the measured latency covers the slowest requests.

Useful expressions

Under the same ___, A measured ___ whereas B measured ___.

The improvement in ___ comes with ___.

We have not measured ___, so that claim remains open.

Cover this until after your first attempt

B has lower median latency in this benchmark, but it uses one hundred sixty megabytes against our one-hundred-twenty-megabyte budget. A is slower at the median and stays within that budget. I would not call B the better fit without addressing memory. Neither result establishes tail latency or performance on unmeasured production workloads.

Second round

The faster result came from a warmed cache in a later rerun. Ask for matched conditions before comparing that rerun with a cold run.

Debrief and extend with AI

Which phrase helped the listener? Which case fact needed clarification? Say one better follow-up question, then repeat the exchange.

ROLE-PLAY CASE 07

Discussing retries and shared state

SHARED CASE

A fictional reservation client times out before receiving a reply. The reservation may already have been created. A colleague proposes automatically submitting a second request as a new reservation. The service's duplicate-request behavior is unspecified; no production change is authorized.

Role A | Responding professional

Explain why repeating a state-changing request needs an explicit behavior contract. Use only the facts given. Choose two relevant terms and be ready to explain one of them in ordinary words.

Role B | Listener

Client developer: you want a responsive experience but do not know whether the server created the reservation before the timeout.

Useful expressions

A missing reply does not establish that ____.

If the first request completed, a new request could ____.

Before retrying, we need agreement on ____.

Cover this until after your first attempt

I would not treat the timeout as proof that no reservation was created. A second request submitted as new could create another reservation if the first one completed. Could we define how repeated requests are recognized and what result the client should receive? We need that contract before recommending an automatic retry for this operation.

Second round

Someone says wrapping work in a transaction automatically settles all duplicate-request behavior. Ask which invariant and concurrency guarantees actually apply.

Debrief and extend with AI

Which phrase helped the listener? Which case fact needed clarification? Say one better follow-up question, then repeat the exchange.

ROLE-PLAY CASE 08

Handing over a release with explicit limits

SHARED CASE

Fictional build R18 is deployed to staging. Its new search view is disabled by a feature flag. A schema migration adds an optional field, but compatibility with the previous application build is untested. Lee takes the release review; production rollout is undecided.

Role A | Responding professional

Give a release handover that distinguishes installed software from enabled behavior and verified recovery. Use only the facts given. Choose two relevant terms and be ready to explain one of them in ordinary words.

Role B | Listener

Lee: accept the compatibility review and ask which environment, build, and feature state the evidence covers.

Useful expressions

Build ___ is deployed to ___; the feature is ___.

The data change is ___, while compatibility with ___ is ___.

Please confirm ownership of ___ before the release decision.

Cover this until after your first attempt

R18 is deployed to staging, with the new search view disabled by its feature flag. The optional-field migration is present there, but compatibility with the previous application build remains untested. Lee, please take that compatibility check into the release review. No production rollout or successful rollback has been confirmed by these staging facts.

Second round

A manager calls the disabled feature fully tested because the build started. Explain which user path has not been exercised.

Debrief and extend with AI

Which phrase helped the listener? Which case fact needed clarification? Say one better follow-up question, then repeat the exchange.

Lesson conversations and speaking practice

The following sections match 05 and 06 on the industry webpage and in the learner workbook. Each lesson has three ten-turn conversations and two bounded speaking scenarios. These are original fictional teaching scripts, not transcripts of real people.

LESSON 01 / 05 · CONVERSATIONS / 1 OF 3

What does null do to a nickname?

Two fictional developers resolve an unspecified input in the profile contract.

01 · Client developer: I want to clear a nickname. Can my client send null for that?

02 · Service developer: The contract specifies an empty string for clearing. Null doesn't currently have a defined meaning.

03 · Client developer: How is an optional field different from a nullable field in this interface contract?

04 · Service developer: Optional allows omission; nullable allows null. Here, omission leaves the nickname unchanged; an empty string clears it.

05 · Client developer: Then null shouldn't silently mean unchanged on one client and cleared on another.

06 · Service developer: Agreed. Shall we propose rejecting null with a validation error until another meaning is agreed?

07 · Client developer: That keeps the current clearing behavior explicit. My client can send an empty string.

08 · Service developer: Please also preserve omission for updates that don't intend to change the nickname.

09 · Client developer: I will. Those are two different user intentions, even if both look blank.

10 · Service developer: Then our proposed contract keeps both outcomes and identifies null as unsupported, pending review.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 01 / 05 • CONVERSATIONS / 2 OF 3

An empty list is a successful result

Fictional developers clarify the difference between no search matches and a failed search.

01 • Consumer developer: Our search adapter treats an empty list as an error. Is that intended?

02 • Interface owner: No. In this contract, an empty list means the search succeeded without finding matches.

03 • Consumer developer: What represents a failed request? I need to show a different message for that.

04 • Interface owner: A validation error has a separate error result. It doesn't use the successful list shape.

05 • Consumer developer: So no matches should display an empty state, while invalid input should explain the error?

06 • Interface owner: Exactly. The interface contract separates those outcomes even though neither produces visible result rows.

07 • Consumer developer: I'll remove the assumption that a successful result must contain at least one item.

08 • Interface owner: Please add the zero-match example beside the populated example when reviewing the adapter.

09 • Consumer developer: That will make the distinction concrete. We can check both without guessing from list length.

10 • Interface owner: And callers can offer another search without telling users that a successful request failed.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 01 / 05 • CONVERSATIONS / 3 OF 3

A renamed response field breaks an older caller

A fictional code review identifies a compatibility concern in a response change.

01 • Reviewer: The new response replaces `displayName` with `label`. What happens to the older caller?

02 • Developer: It still reads `displayName`. I changed the server response but haven't changed that client.

03 • Reviewer: Then this affects backward compatibility, not simply an internal variable name.

04 • Developer: You're right. The field crosses the interface boundary, so consumers can observe the difference.

05 • Reviewer: Do we know whether both fields could temporarily coexist under an agreed transition contract?

06 • Developer: That needs discussion with the client owner. We haven't specified precedence if values differ.

07 • Reviewer: Please keep the response change out of the internal cleanup until that contract is settled.

08 • Developer: I can separate it. The local variable rename doesn't require changing the response field.

09 • Reviewer: That gives us a focused review and keeps the existing caller's expectation intact.

10 • Developer: I'll retain `displayName` in this change and open the proposed interface transition for discussion.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 01 / 06 • SAY IT

Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

Scenario 1 | Clarifying interface contracts

A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Client developer: you want one way to clear a nickname, and your current client can send either an empty string or null.

Second round: A colleague calls omission and an empty string interchangeable. Restate their different outcomes in this fictional contract.

Scenario 2 | A required currency without a default

A fictional pricing interface requires a currency code. A client omits it, assuming dollars. The contract supplies no default. The interface owner can confirm that omission is invalid; no conversion is involved.

Role A: Client developer: ask whether omission has a supported meaning and restate the answer.

Role B: Interface owner: explain that currency is required and a value must be supplied explicitly.

Possible opening: Does an omitted currency mean dollars in this contract, or is it invalid?

Success checks

- Separate example data from the contract.
- Confirm currency must be explicit.
- Avoid inventing a conversion or default behavior.

Add a complication: Someone points to a dollar sign in an example and calls it a default rule.

LESSON 02 / 05 • CONVERSATIONS / 1 OF 3

The empty date is still a hypothesis

Fictional developers discuss a failing three-row import and a passing two-row file.

01 • Developer: The three-row import fails in date parsing. I suspect the row with an empty date.

02 • Colleague: Does your passing example differ only in that date, or in other ways?

03 • Developer: It has two rows instead of three, so the comparison also changes row count.

04 • Colleague: Then it doesn't isolate the empty field. What does the stack trace actually show?

05 • Developer: The call path reaches date parsing before the failure; it doesn't establish the original cause.

06 • Colleague: Could we keep three rows and change only the disputed date in a controlled example?

07 • Developer: Yes. That would distinguish the date hypothesis from the row-count difference in these files.

08 • Colleague: Please preserve the failing input too, so another developer can reproduce the observation.

09 • Developer: I've labeled the cause unconfirmed and attached both existing examples with their differences.

10 • Colleague: Good. We have a specific hypothesis and a comparison that can test it.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 02 / 05 • CONVERSATIONS / 2 OF 3

The error line is not necessarily where the defect began

Fictional developers inspect a trace from a report formatter.

01 • Developer: The trace points to the formatter. Should we assign the whole defect there?

02 • Reviewer: Which value reached it? A failure location doesn't tell us where invalid state originated.

03 • Developer: The formatter expected a title, but the input object contains no title field.

04 • Reviewer: Does the upstream contract require that field, or may callers legitimately omit it?

05 • Developer: It requires a title. The minimal example bypasses the normal builder and leaves it out.

06 • Reviewer: Then we've identified a contract violation in the example, not yet a formatter implementation defect.

07 • Developer: That's useful. The user report may still involve a missing title from the real builder.

08 • Reviewer: Exactly. We need the actual path before deciding whether the example represents that report.

09 • Developer: I'll separate the synthetic missing-field failure from the unconfirmed production report in the discussion.

10 • Reviewer: And I'll review the builder's output contract with you rather than blame the final trace line.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 02 / 05 • CONVERSATIONS / 3 OF 3

One successful rerun doesn't establish a correction

Fictional developers discuss intermittent behavior without claiming a fix.

01 • Maintainer: The import passed on its second attempt. Can we close the failure report?

02 • Developer: No code changed between attempts. A passing rerun doesn't establish that the cause was removed.

03 • Maintainer: Were the input and build the same, or did something change without being noted?

04 • Developer: The file and build match. We didn't capture the external response from the first attempt.

05 • Maintainer: So an external difference is possible, but we don't have evidence identifying one.

06 • Developer: Right. I'll record one failure and one success under those known matching conditions.

07 • Maintainer: Can you retain the first trace instead of replacing it with the successful output?

08 • Developer: Yes. Both outcomes matter, especially because we cannot yet explain their difference.

09 • Maintainer: Then the status is intermittent observation, cause open, and no verified correction.

10 • Developer: Agreed. The next reproduction needs the missing response context, not just another success count.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 02 / 06 • SAY IT

Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

Scenario 1 | Debugging from reproducible evidence

A fictional import fails on a three-row file containing an empty date field. A two-row file without that row succeeds. The stack trace points to date parsing, but no controlled test has isolated the empty field as the cause.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Developer receiving the bug: ask which inputs and conditions differ between the passing and failing examples.

Second round: The same failing file succeeds once after a retry. Explain why one success does not establish a fix.

Scenario 2 | A failure that depends on input order

In a fictional example, records A then B produce a duplicate label, while B then A do not. The same records and build are used. No internal cause has been established.

Role A: Developer: describe the observed ordering difference and propose a controlled comparison.

Role B: Maintainer: ask what is identical and whether the result proves a particular internal defect.

Possible opening: The same two records produce different results when their order changes.

Success checks

- Preserve the order-sensitive observation.
- Keep the internal explanation unconfirmed.
- Agree a smaller order comparison with recorded outputs.

Add a complication: A colleague declares the data corrupted even though the records are unchanged.

LESSON 03 / 05 • CONVERSATIONS / 1 OF 3

A rename that also moves a notification

A fictional author and reviewer resolve an unintended side effect in a pull request.

01 • Reviewer: The helper rename is clear, but moving the notification changes the failed-export path.

02 • Author: I intended only a rename. Which movement changes what the user would see?

03 • Reviewer: The call now sits outside the success branch, so failure would also produce a completion notification.

04 • Author: You're right. That's an unintended side effect, not part of the requested cleanup.

05 • Reviewer: Could you retain the success condition and add a check for the failure path?

06 • Author: Yes. I'll restore that condition and assert that failed exports don't show completion.

07 • Reviewer: The naming choice isn't a blocker. My required change concerns the notification's behavior.

08 • Author: That distinction helps. I'll leave the name and focus the revision on the conditional call.

09 • Reviewer: Once revised, I'll review the changed branch and the new assertion together.

10 • Author: I've accepted that request. This remains a rename with preserved behavior, not a notification redesign.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 03 / 05 • CONVERSATIONS / 2 OF 3

An optional suggestion among required changes

Fictional reviewers distinguish a readability preference from an incorrect boundary check.

01 • Author: You left comments about the variable name and the upper-bound check. Are both blockers?

02 • Reviewer: The boundary check is required: this contract accepts ten, but the new comparison rejects it.

03 • Author: And the name suggestion? I used the same name as the neighboring helper.

04 • Reviewer: That's optional. I find limit clearer, but the existing naming convention is defensible.

05 • Author: I'll correct the boundary and keep the established name to avoid unrelated changes.

06 • Reviewer: That works. Please include the accepted value ten and the rejected value eleven in the evidence.

07 • Author: Those examples make the intended boundary clear without expanding this into a style cleanup.

08 • Reviewer: Exactly. My review should distinguish a behavior mismatch from a preference about readability.

09 • Author: I'll mark the naming thread as discussed and return the boundary revision for review.

10 • Reviewer: Then the required action is unambiguous, and the optional comment doesn't become an accidental condition.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 03 / 05 • CONVERSATIONS / 3 OF 3

A helper extracts more behavior than expected

Fictional developers discuss a refactor that unexpectedly reorders an observable event.

01 • Reviewer: The extracted helper sends the receipt before the save completes. Was that ordering change intentional?

02 • Author: No. I moved both operations into one helper but changed their order accidentally.

03 • Reviewer: The existing behavior sends only after a successful save. Callers may rely on that distinction.

04 • Author: Refactoring should preserve that observable sequence, not merely reuse the same calls.

05 • Reviewer: Could you keep the original ordering and explain the helper's responsibility more narrowly?

06 • Author: Yes. It will coordinate the existing sequence rather than introduce a new notification policy.

07 • Reviewer: Please keep the unrelated retry proposal outside this pull request so we can review each behavior separately.

08 • Author: I'll separate that proposal. The current revision will only extract the unchanged sequence.

09 • Reviewer: That gives the tests a clear expectation: no receipt before the successful save outcome.

10 • Author: And it makes the refactor claim something we can actually check against observable behavior.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 03 / 06 • SAY IT

Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

Scenario 1 | Giving actionable code-review feedback

A fictional pull request is titled 'rename export helper.' Its diff also moves a notification call outside a success branch, so failed exports would now notify the user as if complete. The naming change is acceptable; the behavior change is unintended.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Author: the rename is intentional, but moving the notification was accidental. Ask which path needs a check.

Second round: A second reviewer adds an optional naming preference. Label it optional while keeping the failure-path correction clearly required.

Scenario 2 | A review comment that needs an example

A fictional review says 'This loses precision.' The author rounded a displayed duration, but the underlying calculation remains unchanged. The reviewer is concerned only about the displayed boundary between 0.9 and 1.0 seconds.

Role A: Author: ask which value, output, and consequence the comment refers to.

Role B: Reviewer: give the 0.9-to-1.0 display example and distinguish it from calculation accuracy.

Possible opening: Could you identify which output loses precision and whether your concern is display or calculation?

Success checks

- Ask for the concrete example.
- Separate presentation from underlying computation.
- Agree a focused review of display rounding.

Add a complication: A teammate proposes rewriting the calculation before the display concern is clarified.

LESSON 04 / 05 • CONVERSATIONS / 1 OF 3

The package we didn't list directly

Fictional developers identify evidence needed for a ParcelKit upgrade review.

01 • Reviewer: ParcelKit is our direct dependency; what introduces TextCore into the proposed build?

02 • Maintainer: TextCore is a transitive dependency through ParcelKit. Its resolved version changes in the proposed lockfile.

03 • Reviewer: Could you show that difference and link the note about the deprecated parsing entry point?

04 • Maintainer: I have both. I haven't checked whether our application or ParcelKit uses that entry point.

05 • Reviewer: Please identify the caller before describing this as an update requiring no code changes.

06 • Maintainer: Agreed. Deprecation doesn't automatically mean removal, but the actual call and version still matter.

07 • Reviewer: Exactly. We need evidence about this combination, not an assumption based on the package name.

08 • Maintainer: I'll return the dependency path, resolved versions, change note, and relevant call-site findings.

09 • Reviewer: That will let us separate package resolution from compatibility with the calls we make.

10 • Maintainer: And we'll leave the drop-in claim unconfirmed until those findings support it.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 04 / 05 • CONVERSATIONS / 2 OF 3

A lockfile difference without a source change

Fictional maintainers discuss why two builds resolved different dependency sets.

01 • Developer: My source changes only a label, yet the proposed build resolves three different package versions.

02 • Maintainer: Did the dependency manifest change, or only the lockfile that records this resolution?

03 • Developer: Only the lockfile differs. I don't know which tool action regenerated it locally.

04 • Maintainer: Then the label change and dependency change need separate explanations before we call the build equivalent.

05 • Developer: Could I describe the lockfile as generated and therefore irrelevant to behavior?

06 • Maintainer: No. Generated doesn't mean inconsequential; the resolved dependencies may affect what goes into the artifact.

07 • Developer: I'll identify how it changed and compare the selected versions with the current build.

08 • Maintainer: Please check the actual tool documentation rather than assume every package manager treats lockfiles identically.

09 • Developer: Understood. The source edit remains narrow, but the artifact difference is not yet explained.

10 • Maintainer: That's the right scope for review: one known label change and an open resolution difference.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 04 / 05 • CONVERSATIONS / 3 OF 3

Deprecated is not removed

A fictional maintainer clarifies an upgrade note with a caller's developer.

01 • Caller developer: The note says the old reader is deprecated. Does our current call stop working immediately?

02 • Maintainer: The note doesn't establish removal. It says the replacement is preferred, with removal timing unspecified.

03 • Caller developer: So we should distinguish a migration recommendation from a confirmed missing entry point.

04 • Maintainer: Yes. We also need to compare the replacement's input and error behavior before changing callers.

05 • Caller developer: Our current caller expects an error result for malformed input. Does the replacement preserve that?

06 • Maintainer: The note links a contract section, but I haven't checked that specific behavior yet.

07 • Caller developer: Please include it in the review. A matching function name wouldn't settle that difference.

08 • Maintainer: I'll compare the stated contracts and identify any behavior our caller relies on.

09 • Caller developer: Then today's conclusion is deprecated use identified, replacement compatibility still under review.

10 • Maintainer: Exactly. We can discuss a migration without inventing an immediate failure or a removal deadline.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 04 / 06 • SAY IT

Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

Scenario 1 | Requesting dependency evidence

A fictional application directly uses package ParcelKit, which brings in TextCore indirectly. A proposed ParcelKit upgrade changes the resolved TextCore version. The change note mentions a deprecated parsing entry point; the team's usage and replacement compatibility are unknown.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Maintainer: you have the change note and resolved dependency list, but have not searched the application's call sites yet.

Second round: The package name is unchanged. Explain why that alone does not establish identical behavior across versions.

Scenario 2 | Two components request different versions

In a fictional dependency report, App requests FormatKit 1.4; App -> ViewPlug requests FormatKit 2.1. The supplied graph retains both versions at those paths; the tool's resolution rules remain unreviewed.

Role A: Developer: ask what was actually resolved and where each request originates.

Role B: Maintainer: show the two requests and state that the tool's resolution rules need checking.

Possible opening: These are two requested versions; which versions actually appear in the resolved graph?

Success checks

- Distinguish requested from resolved versions.
- Identify both dependency paths.
- Avoid claiming a universal resolution rule.

Add a complication: Someone assumes the higher version always wins in every package tool.

LESSON 05 / 05 • CONVERSATIONS / 1 OF 3

The storage substitute never rejects

Fictional developers examine why a unit test missed a misleading success message.

01 • Reviewer: The save test passed before the bug report. What outcome did its storage substitute return?

02 • Developer: It always returned success, and the assertion checked that the success message appeared.

03 • Reviewer: Then it never exercised the rejection that previously produced the wrong message, did it?

04 • Developer: No. We need a regression case where the substitute rejects and the visible result is checked.

05 • Reviewer: Would asserting only that the function returns without throwing catch the misleading success message?

06 • Developer: Not necessarily. The assertion needs to distinguish the intended rejection message from false success.

07 • Reviewer: Good. And what would that case tell us about the real storage adapter?

08 • Developer: Only about the save function's response to the simulated outcome, not the adapter's actual behavior.

09 • Reviewer: Then name the integration question separately so the unit test isn't credited with answering it.

10 • Developer: The unit check covers rejection response; an integration test must examine the real adapter.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 05 / 05 • CONVERSATIONS / 2 OF 3

A test that calls code but checks no result

Fictional developers strengthen the stated purpose of a parsing test.

01 • Developer: This new test calls the parser with a valid record and completes without an exception.

02 • Reviewer: What behavior is it intended to establish? Execution alone doesn't check the parsed values.

03 • Developer: It should preserve leading zeros in a reference, but I haven't asserted that result.

04 • Reviewer: Then a parser returning a different reference might pass this test as currently described.

05 • Developer: You're right. The concrete input is zero zero seven, and the expected reference retains all three characters.

06 • Reviewer: Please make that output the check, rather than relying on successful execution as a substitute.

07 • Developer: That also explains why converting the reference to a number would violate this contract.

08 • Reviewer: Exactly. The test's purpose is preserving an identifier, not checking arithmetic on its digits.

09 • Developer: I'll add the expected reference and name the case around preserving leading zeros.

10 • Reviewer: Then a failure will point to the behavior we care about instead of an unspecified parser problem.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 05 / 05 • CONVERSATIONS / 3 OF 3

A test double with the wrong contract

Fictional developers find a mismatch between a controlled substitute and a real collaborator.

01 • Reviewer: The test double returns a plain string, but the collaborator's contract returns a result object.

02 • Developer: I simplified it because the function only needs the label inside that object.

03 • Reviewer: Does the function actually extract the label, or does the simplified substitute hide that step?

04 • Developer: It extracts the label in production. The test currently bypasses that shape difference.

05 • Reviewer: Then the substitute is not exercising the interface the function is supposed to consume.

06 • Developer: I'll make its returned shape match the contract and keep the sample label simple.

07 • Reviewer: Please include the error result shape too if this case claims to cover rejection.

08 • Developer: This case covers success only. I'll label it narrowly and add rejection as a separate case.

09 • Reviewer: That separates test data simplicity from changing the collaborator's promised interface.

10 • Developer: Agreed. A controlled substitute can be predictable without teaching the test the wrong contract.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 05 / 06 • SAY IT

Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

Scenario 1 | Explaining what a test actually exercises

A fictional save function's unit test replaces storage with a test double that always reports success. It checks the success message but never exercises a storage rejection. A previous bug incorrectly showed success after rejection; the real storage adapter is not tested here.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Reviewer: ask which collaborator is substituted and what assertion would fail if the success message reappeared on rejection.

Second round: A teammate proposes checking only that no exception was thrown. Explain why the visible message needs its own assertion.

Scenario 2 | A test name overstates its assertion

A fictional test named 'saves and reloads settings' checks only the save response. It uses a test double and never performs a reload. The author can rename it and propose a distinct persistence check.

Role A: Reviewer: explain the mismatch between the test name and observed actions.

Role B: Author: distinguish a response check from a read-after-save interaction.

Possible opening: This test checks the save response, but where does it reload the settings?

Success checks

- State exactly what is asserted.
- Avoid treating a response as proof of persistence.
- Agree a truthful name and a separate interaction question.

Add a complication: A teammate says the positive response makes the reload check redundant.

LESSON 06 / 05 • CONVERSATIONS / 1 OF 3

Faster at the median, outside the memory budget

Fictional developers compare implementations A and B under the supplied benchmark.

01 • Developer: B's median latency is twenty-five milliseconds, versus forty for A. Should we choose B?

02 • Reviewer: How does each memory footprint compare with our stated budget?

03 • Developer: A uses eighty megabytes; B uses one hundred sixty. Our budget is one hundred twenty.

04 • Reviewer: Then B improves that latency measure but exceeds the resource constraint we're working within.

05 • Developer: Agreed. A fits the memory budget, though its median is fifteen milliseconds slower.

06 • Reviewer: Do these results tell us about the slowest requests or only the median?

07 • Developer: Only the median. We haven't measured tail latency or representative production workloads yet.

08 • Reviewer: Please include those limits instead of describing B as universally faster and therefore better.

09 • Developer: I'll present A as within budget and B as requiring a memory tradeoff we haven't accepted.

10 • Reviewer: That comparison gives us a decision constraint, not just the most attractive single number.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 06 / 05 • CONVERSATIONS / 2 OF 3

Throughput rises while individual waits grow

Fictional developers interpret two different measures in a batching experiment.

01 • Developer: The batched version completes more items per minute, but individual items wait longer before processing.

02 • Colleague: Then improved throughput doesn't necessarily mean each person gets a faster response, does it?

03 • Developer: Correct. Throughput measures completed work over time; latency measures the wait for a specified item.

04 • Colleague: Which goal matters for this feature? The current requirement is a response within the stated interaction window.

05 • Developer: Then we need the latency evidence against that window, not only the items-per-minute total.

06 • Colleague: Can we show both metrics for the same workload so the tradeoff is visible?

07 • Developer: Yes. The benchmark includes both, although we haven't measured the busiest production pattern.

08 • Colleague: Please keep that limitation. A larger batch might help total capacity while making this interaction less responsive.

09 • Developer: I'll explain the measured tradeoff without presenting higher throughput as a synonym for lower latency.

10 • Colleague: And we'll ask whether the observed wait meets the actual requirement before favoring the batch.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 06 / 05 • CONVERSATIONS / 3 OF 3

A warm run beside a cold run

Fictional developers correct an unfair benchmark comparison.

01 • Reviewer: Your chart compares A's first run with B's fifth run. Were their starting conditions equivalent?

02 • Developer: No. B reused initialized data, while A's first run included preparing that data.

03 • Reviewer: Then we can't attribute the entire timing difference to the implementation change alone.

04 • Developer: Agreed. The warm and cold conditions need separate comparisons using matching workloads.

05 • Reviewer: Can you label the existing results honestly instead of discarding the slower observation?

06 • Developer: Yes. I'll mark them cold A and warm B, without claiming a controlled head-to-head result.

07 • Reviewer: What comparison would answer whether B changes the first-use experience for this workload?

08 • Developer: Cold runs for both implementations under the same stated preparation conditions would address that question.

09 • Reviewer: And warm runs for both would answer the repeated-use question, which is different.

10 • Developer: Exactly. I'll keep the two questions separate so a cache advantage doesn't become an unsupported implementation claim.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 06 / 06 • SAY IT

Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

Scenario 1 | Comparing performance tradeoffs fairly

In a fictional controlled benchmark on the same input and machine, implementation A has median latency 40 milliseconds and memory use 80 megabytes; B has 25 milliseconds and 160 megabytes. The available memory budget is 120 megabytes. Tail latency and production workloads are unmeasured.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Reviewer: prioritize the fixed memory budget and ask whether the measured latency covers the slowest requests.

Second round: The faster result came from a warmed cache in a later rerun. Ask for matched conditions before comparing that rerun with a cold run.

Scenario 2 | The average hides the slow end

A fictional benchmark reports unchanged average latency, but its slowest measured group takes longer after a change. Sample size is forty runs; production representativeness is unknown.

Role A: Developer: explain how an unchanged average can coexist with slower individual observations.

Role B: Reviewer: ask for the distribution and comparable conditions, without assuming every user is slower.

Possible opening: The average is unchanged, but the slow end of this sample has moved.

Success checks

- Distinguish the average from the distribution.
- Keep the forty-run scope visible.
- Avoid claiming a production-wide effect.

Add a complication: A stakeholder wants to say there is no performance change because the average matches.

LESSON 07 / 05 • CONVERSATIONS / 1 OF 3

The reply timed out, not necessarily the reservation

Fictional developers discuss a proposed automatic repeat of a state-changing request.

01 • Client developer: The reservation request timed out. I'll send a new one so the user can continue.

02 • Service developer: Do we know the first request failed, or only that the client didn't receive its reply?

03 • Client developer: Only the missing reply. The reservation may already exist, and repeated-request behavior is unspecified.

04 • Service developer: Then another new request could create an additional reservation rather than complete the original interaction.

05 • Client developer: We need a defined way to recognize repetition and return the appropriate outcome.

06 • Service developer: Yes. Idempotency concerns the intended effect of repeating the operation, not identical timing or wording.

07 • Client developer: Could we discuss that contract before proposing an automatic retry for this client?

08 • Service developer: Certainly. We should also state how the client distinguishes an unknown outcome from confirmed rejection.

09 • Client developer: I'll keep the current report at outcome unknown and withdraw the assumption that nothing happened.

10 • Service developer: That gives us the right design question without inventing a production retry mechanism.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 07 / 05 • CONVERSATIONS / 2 OF 3

Two callers each see the last slot

Fictional developers discuss an overlapping-operation example without choosing a database technique.

01 • Developer: Two callers each see one slot available and both receive a successful reservation in this example.

02 • Reviewer: What invariant should hold? We need the intended rule before judging the observed interleaving.

03 • Developer: The agreed rule permits only one active reservation for that slot, so this outcome violates it.

04 • Reviewer: Then the issue depends on overlapping actions, not simply whether either caller behaves correctly alone.

05 • Developer: Right. Each reads availability before either completes the state change in the illustrated sequence.

06 • Reviewer: Calling the operations transactions doesn't tell us which concurrency guarantees this design actually provides.

07 • Developer: Agreed. We need the system's specified guarantees and evidence that the invariant survives overlapping requests.

08 • Reviewer: Please describe the conflicting outcomes and the invariant without prescribing a database-specific command here.

09 • Developer: I'll describe the race condition and violated invariant, with the implementation remedy still under review.

10 • Reviewer: That gives the review a precise invariant, not a vague request for safety.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 07 / 05 • CONVERSATIONS / 3 OF 3

A repeated command is not a new intention

Fictional developers clarify a request identifier's meaning in a proposed contract.

01 • Client developer: Should the client create a new request identifier each time it repeats the same submitted action?

02 • Interface owner: In our proposed contract, repetition of that action retains its identifier; a genuinely new action gets another.

03 • Client developer: Then the identifier distinguishes another attempt from another user intention, not simply another network message.

04 • Interface owner: Exactly. But that is our proposal, not an assumption about every service or library.

05 • Client developer: What if the same identifier arrives with different contents? We haven't defined that case.

06 • Interface owner: Good catch. The contract needs an explicit outcome rather than silently treating different contents as equivalent.

07 • Client developer: I'll add that unresolved case and keep the repetition design marked incomplete.

08 • Interface owner: Please also explain how callers learn an earlier outcome after losing its reply.

09 • Client developer: That links the identifier rule to the actual user problem instead of treating it as decoration.

10 • Interface owner: And it keeps duplicate effects, changed contents, and missing replies as distinct questions to resolve.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 07 / 06 • SAY IT

Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

Scenario 1 | Discussing retries and shared state

A fictional reservation client times out before receiving a reply. The reservation may already have been created. A colleague proposes automatically submitting a second request as a new reservation. The service's duplicate-request behavior is unspecified; no production change is authorized.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Client developer: you want a responsive experience but do not know whether the server created the reservation before the timeout.

Second round: Someone says wrapping work in a transaction automatically settles all duplicate-request behavior. Ask which invariant and concurrency guarantees actually apply.

Scenario 2 | Two editors save different versions

In a fictional editor, two people open revision 6. One saves revision 7. The other's later save still refers to revision 6. Conflict handling is unspecified; neither edit is known to be dispensable.

Role A: Developer: describe the competing updates and ask what the interface should communicate.

Role B: Reviewer: insist that the intended conflict outcome be defined before choosing a mechanism.

Possible opening: The second save is based on an older revision; what outcome should the caller expect?

Success checks

- Identify the older base revision.
- Avoid inventing a universal last-writer rule.
- State the unresolved contract question about competing edits.

Add a complication: A colleague assumes the later request must always overwrite the earlier one.

LESSON 08 / 05 • CONVERSATIONS / 1 OF 3

R18 is installed but search is disabled

A fictional developer hands the release review to Lee.

01 • Developer: R18 is in staging, but the feature flag keeps the new search view disabled.

02 • Lee: Then deployment alone doesn't establish that users have exercised the new view in staging.

03 • Developer: Correct. The optional-field migration is present, but previous-build compatibility with it remains untested.

04 • Lee: Has anyone confirmed production rollout or a successful rollback based on this deployment?

05 • Developer: No. Both would overstate what we know. The open compatibility check belongs in your release review.

06 • Lee: I'll take that check. Please identify the exact build and the staging evidence I should review.

07 • Developer: The handover links R18's artifact and the migration record, with search explicitly marked disabled.

08 • Lee: I accept the review, not a release approval. The production decision is still open.

09 • Developer: Thank you. Application rollback also shouldn't be described as automatically undoing the schema change.

10 • Lee: Agreed. I'll keep build state, feature exposure, and recovery evidence separate in the decision.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 08 / 05 • CONVERSATIONS / 2 OF 3

The artifact differs from the one tested

Fictional developers catch a release handover mismatch.

01 • Release reviewer: The release note names build K9, but the test evidence is labeled K8. Are they equivalent?

02 • Developer: They're different build artifacts. I haven't established that their contents are identical.

03 • Release reviewer: Then we can't attach K8's passing results to K9 without explaining that difference.

04 • Developer: Agreed. The intended release artifact needs its own identified evidence or a justified equivalence check.

05 • Release reviewer: What changed between the runs? The source label looks the same in the note.

06 • Developer: The dependency resolution may differ. Matching source labels alone don't settle the artifact contents.

07 • Release reviewer: Please keep the release candidate unverified until that build difference is explained and reviewed.

08 • Developer: I'll correct the evidence link's description so it says K8, rather than implying K9 was tested.

09 • Release reviewer: That protects the handover from a mistaken identity, without discarding useful results from K8.

10 • Developer: And it gives the next reviewer a precise artifact question instead of a generic green status.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 08 / 05 • CONVERSATIONS / 3 OF 3

Switching a flag off doesn't erase prior changes

Fictional developers clarify the scope of disabling a feature after a trial.

01 • Release lead: The flag is off again. Does that mean the trial's stored changes are gone?

02 • Developer: No. Our flag controls the view, not reversal of stored data from the trial.

03 • Release lead: So disabling exposure and undoing data changes are separate actions with separate evidence.

04 • Developer: Exactly. We can confirm the view is disabled, but we haven't confirmed any data reversal.

05 • Release lead: Could the previous application build still read the data written during the trial?

06 • Developer: That compatibility question remains open. We shouldn't infer the answer from the flag's current state.

07 • Release lead: Please make that limit clear in the handover so support doesn't promise restored data.

08 • Developer: I'll state view disabled, trial data retained, and previous-build compatibility awaiting review.

09 • Release lead: That's specific enough for the review. We need an owner for the compatibility question.

10 • Developer: I'll bring it to the maintainer and retain the open status until someone accepts the check.

Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

LESSON 08 / 06 • SAY IT

Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

Scenario 1 | Handing over a release with explicit limits

Fictional build R18 is deployed to staging. Its new search view is disabled by a feature flag. A schema migration adds an optional field, but compatibility with the previous application build is untested. Lee takes the release review; production rollout is undecided.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Lee: accept the compatibility review and ask which environment, build, and feature state the evidence covers.

Second round: A manager calls the disabled feature fully tested because the build started. Explain which user path has not been exercised.

Scenario 2 | A rollout pauses after a bounded trial

A fictional feature is enabled for ten internal test accounts. Two report missing labels. The release lead pauses further exposure; existing evidence covers only those ten accounts and does not establish an external impact.

Role A: Developer: report the observed failures, exposure boundary, and paused expansion.

Role B: Release reviewer: ask what is known about the other eight accounts and external users.

Possible opening: Two of ten internal test accounts reported missing labels, and further exposure is paused.

Success checks

- Preserve the ten-account scope.
- Distinguish reports from complete checks of the other accounts.
- Name the remaining investigation without expanding the impact claim.

Add a complication: A stakeholder asks to describe every user as affected.

Give feedback that helps

Score each criterion 0, 1, or 2. This is a practice rubric, not a professional qualification or CEFR test. Do not award or remove points for a particular accent.

Meaning and accuracy

Keeps the case facts accurate; clearly separates confirmed and unknown information.

Clarity and language

Uses understandable sentences and explains specialist terms when the listener needs it.

Interaction and tone

Responds to the other person's question, checks understanding, and uses an appropriate tone.

Task completion

Makes the requested action or unresolved question clear without inventing authority or facts.

Use the same scale for each criterion

0 - Not yet: The reader or listener cannot yet recover the intended meaning or action.

1 - With support: The message works after a prompt, clarification, or revision.

2 - Independently: The message is clear and accurate without a prompt.

Feedback sequence

First, identify a successful phrase. Next, identify one point where meaning or accuracy needs work. Offer one useful language correction, allow a repeat, and compare the two attempts. A total out of eight describes this performance only; do not translate it into a proficiency level.

Final challenge

Choose a case not rehearsed today. The learner gives a one-minute response, answers two follow-up questions, and switches roles and repeats the exchange. Add the lesson's second-round challenge. Compare the result with an earlier attempt using the four criteria.

Use the language responsibly

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

Meaning before performance

Use specialist terms only when they help the listener. Explain unfamiliar words, preserve uncertainty, and ask when an instruction or responsibility is unclear. The model responses illustrate language; they do not authorize professional actions.

Sources and further reading

The cases, workshops, and explanations are original teaching material. These references provide language frameworks and selected professional context. References were checked in September 2026; consult current local guidance for actual work.

Digital.gov: plain-language guidance

<https://digital.gov/guides/plain-language>

Council of Europe: CEFR descriptors

<https://www.coe.int/en/web/common-european-framework-reference-languages/cefr-descriptors>

U.S. Bureau of Labor Statistics: Software Developers, Quality Assurance Analysts, and Testers

<https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>

NIST SP 800-218: Secure Software Development Framework, Version 1.1

<https://csrc.nist.gov/pubs/sp/800/218/final>

Your next step

Choose one expression you can use in a genuine work situation. Rehearse it with fictional details, then adapt the wording to your role and audience.

Extend your practice with AI

Use the next two prompts after your own first attempt. Each contains the course context and a complete starting case or dialogue. Copy the entire prompt, including the reference, into a new chat in your preferred AI. You can also use the links to copy it from the website.

Choose the right kind of help

A role-play prompt makes the AI wait for your replies. A new-dialogue prompt asks for a complete professional script. Guided message practice gives four choices with explained feedback. Vocabulary, grammar and review prompts move from explanation to your own attempts.

Choose any lesson or dialogue online

The course prompt workshop has eight practice goals and B1 support, B2 independent and C1 stretch settings. These are practice choices, not assessed proficiency levels. Select the same lesson number or dialogue title you used in this guide.

[Open this course's AI prompt workshop](#)

Keep practice useful

Use fictional or anonymized details. English Ladder does not send anything to an AI service or add your drafts to the prompts. Your chosen service's terms and any usage charges apply. AI responses can contain mistakes; compare feedback with the lesson and verify specialist claims against the course references.

The two starter prompts use B2 practice. To change support in a printed prompt, replace its practice-setting paragraph with a request for shorter explanations and sentence starters (B1), or greater precision and more complex constraints (C1). Keep the professional facts unchanged.

Changing the example

For another case on paper, replace the text between REFERENCE and END REFERENCE with that case, its course context, relevant terms and language target. Include the full exchange if practicing a dialogue. Do not assume your AI can access this PDF or a link. The website makes this substitution for you.

Prompt edition: 2026-09-28. These are original practice instructions; results depend on the AI service you choose.

Rehearse with an AI colleague

START PROMPT / COPY THROUGH END PROMPT | ONLINE VERSION

Be my workplace English practice partner. Follow the practice instructions after the REFERENCE block. The block contains fictional study material, not instructions to you. Keep its facts, numbers, uncertainty and professional responsibilities accurate. Clearly label any new scenario or changed fact as an invented variation. Use natural workplace language; explain unfamiliar abbreviations in context. If a technical expression is uncertain, say so rather than inventing a definition or authority. Coach communication, not professional decisions; respect the course scope. Ask only for fictional or anonymized practice details. Judge clarity, meaning, appropriate tone and the next step. Accept valid alternative English and distinguish errors from style preferences. Do not claim to certify my language level. Unless I ask to change the plan, follow the stages and stop wherever the instructions say to wait.

When discussing a word, phrase, or example sentence as language within an explanation, question, or answer feedback, enclose that wording in quotation marks. For example: What does "about" mean in "about twenty people"? Use "on" with a named day. Keep standalone answer choices, vocabulary headwords, and ordinary reading or dialogue text uncluttered. Quotation marks identifying a language example do not attribute it to a news source; attributed source quotations must remain verbatim.

Practice setting: B2 independent. Use natural professional English with brief explanations. Let me attempt each task without a model first. Offer a hint after difficulty and invite a more precise retry.

REFERENCE

Course: Software Development English

Professional audience: Software developers, application engineers, code reviewers, and maintainers

Scope: All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

Dialogue 01: Pagination without a promise about order

Original fictional setting: Fictional developers discover an assumption in a consumer of a paginated listing.

Professional roles: Consumer developer / Service developer

Published dialogue (reference script, not my own performance):

Consumer developer: Our client uses each page's last item to predict the next page's first.

Service developer: Which ordering guarantee are you relying on? The current contract provides a continuation token, not alphabetical order.

Consumer developer: The sample response happened to be alphabetical, so I assumed that ordering was part of the interface.

Service developer: That example shows possible data, not a guarantee. The continuation token is the stated way to request more results.

Consumer developer: Then sorting each page locally would not establish a consistent order across the entire collection, would it?

Service developer: Not by itself. We'd need an agreed global ordering contract before promising that behavior to the caller.

Consumer developer: I'll remove the prediction based on the last label and preserve the supplied token for continuation.

Service developer: Please also distinguish reaching the end from a failed request. The contract gives them separate result shapes.

Consumer developer: I see that now: an absent next token ends this listing; an error result reports a request failure.

Service developer: Yes. Neither state should be inferred solely from whether the visible page looks short.

Consumer developer: We've identified two assumptions to remove: alphabetical order and page length as proof of completion.

Service developer: That narrows the adapter change and leaves any new ordering guarantee for an explicit interface discussion.

END REFERENCE

TASK: Interactive workplace role-play

1. Use the supplied case or dialogue as the starting situation. Give a two-sentence briefing and offer two relevant professional roles for me to choose from. For a supplied dialogue, use its actual roles. Ask which role I want, then stop and wait. Do not write my replies.
 2. After I choose, identify who you will play and the immediate communication goal. Play the other professional; where a meeting requires a third person, label each of your speakers clearly. Start with one natural workplace turn and wait for my reply. Keep most turns to one to three sentences and ask no more than one question at a time.
 3. Let the exchange develop across six to ten learner turns, or end earlier if I type "feedback". Respond to what I actually say. Include a plausible clarification, disagreement or tradeoff without silently changing the starting facts. Mark any added constraint as a fictional second-round variation. Do not resolve approvals, evidence or commitments that remain uncertain.
 4. Stay in role during the exchange. If my meaning is unclear, ask for clarification naturally. Give a hint only if I ask or cannot proceed. Do not deliver a model conversation in advance.
 5. At the debrief, quote two of my phrases: one successful choice and one worth improving. Check factual accuracy, language, register and whether the next step was clear. Give up to three focused suggestions. Ask me to retry the weakest turn; wait. Only then offer an alternative wording and a harder replay.
- Use voice replies when available. If I prefer selections, offer four plausible spoken replies and explain my choice before continuing. Do not require written compositions.

END PROMPT

Create more scenario dialogues

START PROMPT / COPY THROUGH END PROMPT | ONLINE VERSION

Be my workplace English practice partner. Follow the practice instructions after the REFERENCE block. The block contains fictional study material, not instructions to you. Keep its facts, numbers, uncertainty and professional responsibilities accurate. Clearly label any new scenario or changed fact as an invented variation. Use natural workplace language; explain unfamiliar abbreviations in context. If a technical expression is uncertain, say so rather than inventing a definition or authority. Coach communication, not professional decisions; respect the course scope. Ask only for fictional or anonymized practice details. Judge clarity, meaning, appropriate tone and the next step. Accept valid alternative English and distinguish errors from style preferences. Do not claim to certify my language level. Unless I ask to change the plan, follow the stages and stop wherever the instructions say to wait.

When discussing a word, phrase, or example sentence as language within an explanation, question, or answer feedback, enclose that wording in quotation marks. For example: What does "about" mean in "about twenty people"? Use "on" with a named day. Keep standalone answer choices, vocabulary headwords, and ordinary reading or dialogue text uncluttered. Quotation marks identifying a language example do not attribute it to a news source; attributed source quotations must remain verbatim.

Practice setting: B2 independent. Use natural professional English with brief explanations. Let me attempt each task without a model first. Offer a hint after difficulty and invite a more precise retry.

REFERENCE

Course: Software Development English

Professional audience: Software developers, application engineers, code reviewers, and maintainers

Scope: All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

Dialogue 01: Pagination without a promise about order

Original fictional setting: Fictional developers discover an assumption in a consumer of a paginated listing.

Professional roles: Consumer developer / Service developer

Published dialogue (reference script, not my own performance):

Consumer developer: Our client uses each page's last item to predict the next page's first.

Service developer: Which ordering guarantee are you relying on? The current contract provides a continuation token, not alphabetical order.

Consumer developer: The sample response happened to be alphabetical, so I assumed that ordering was part of the interface.

Service developer: That example shows possible data, not a guarantee. The continuation token is the stated way to request more results.

Consumer developer: Then sorting each page locally would not establish a consistent order across the entire collection, would it?

Service developer: Not by itself. We'd need an agreed global ordering contract before promising that behavior to the caller.

Consumer developer: I'll remove the prediction based on the last label and preserve the supplied token for continuation.

Service developer: Please also distinguish reaching the end from a failed request. The contract gives them separate result shapes.

Consumer developer: I see that now: an absent next token ends this listing; an error result reports a request failure.

Service developer: Yes. Neither state should be inferred solely from whether the visible page looks short.

Consumer developer: We've identified two assumptions to remove: alphabetical order and page length as proof of completion.

Service developer: That narrows the adapter change and leaves any new ordering guarantee for an explicit interface discussion.

END REFERENCE

TASK: Extend the conversation library

1. Propose six distinct, common scenarios in this field beyond the supplied case or script. For each, name the setting, two or more professional roles, the communication problem and one useful terminology focus. Include routine coordination, clarification, a complication, disagreement, handoff and follow-up where relevant. Choose scenarios that fit this profession, rather than forcing unsuitable situations into it.
2. Ask me to choose one scenario, or request all six in sequence. Stop and wait. Do not write all the scripts before I choose.
3. For the selected scenario, write an original fictional dialogue of 12-18 substantial speaking turns between two or three professionals. Name each role, establish an actual work problem, and let the exchange progress through questions, clarification, competing constraints and a credible next step or explicitly unresolved issue. Use the occupation's natural nomenclature and register. Avoid an interview between a teacher and a learner, generic small talk, and inserting a glossary definition into every reply. Explain specialist terms outside the dialogue instead.
4. After the script, explain five useful expressions in context, identify two grammar or register choices and ask three questions about the speakers' reasoning. Withhold the answers until I attempt them. Add one role-switch challenge.
5. Before presenting a script, check names, numbers, chronology, roles and terminology for consistency. Label invented facts and acknowledge uncertain specialist usage. If I requested all six, deliver one complete script at a time and wait for "next". Make each scenario materially different.

END PROMPT